

MÓDULO V

Introdução à Programação II

MÓDULO V

O que é Versionamento e para que serve?

O versionamento é uma maneira organizada de administrar mudanças, criando versões.

Seja em um documento de texto ou software, a cada incremento, teremos uma nova versão daquilo que estamos criando, pois estamos inserindo algo que não estava lá anteriormente.

Nesse sentido, o versionamento atua criando um histórico das mudanças realizadas, que pode ser acessado e utilizado para recuperar alguma versão específica.

MÓDULO V

O que é uma ferramenta de Versionamento?

Tratam-se de **ferramentas que administram todo o ciclo de vida do trabalho.**

Elas são responsáveis por gerar um histórico contendo todas as alterações realizadas no arquivo ou documento. Dados como nome da pessoa responsável, data e hora da modificação também são registrados.

Além disso, as ferramentas de versionamento permitem ainda que outras pessoas possam acessar e modificar o trabalho simultaneamente e que ramificações da versão principal sejam criadas em paralelo.

MÓDULO V

Quais são as 5 melhores ferramentas de controle de versão de software?

1. CVS

A CVS é uma das ferramentas de controle de software mais antigas no mercado. A primeira versão dela foi desenvolvida em 1968. Essa ferramenta possui como maior desvantagem o fato de ser considerada como uma tecnologia antiga. Porém, ainda é bastante utilizada por equipes de desenvolvedores.

É muito simples de ser operada. Isso significa que a sua equipe pode aprender rapidamente como usar todas as funcionalidades da CVS com eficiência.

MÓDULO V

2. Subversion

No meio corporativo, o Subversion é uma ferramenta de controle de versão de software bastante utilizada. Ela é bastante rápida na execução das funcionalidades do sistema e ainda se mostra como uma das mais simples de ser empregada. Isso significa que com um conhecimento básico de conceitos relacionados ao controle de versão de software é possível executar comandos na ferramenta.

A aprendizagem da equipe também é rápida nesse aspecto.

Um dos problemas do Subversion são as críticas relacionadas à eficácia do software. No passado, essa ferramenta apresentou problemas na hora de executar as principais funções de um controle de versão de software eficiente. Porém, as últimas versões lançadas parecem ter solucionado tudo que foi apontado como desvantagem do programa.

Por fim, o Subversion é uma ferramenta de controle de versão centralizada. Isso significa que não é indicada para todas as equipes de TI, apenas para aquelas que são menores (com apenas algumas dezenas de desenvolvedores) e estão reunidas em um mesmo espaço físico.

MÓDULO V

3. TFS

O **TFS** — (*Team Foundation Server*) — é uma outra ferramenta de controle de versão de software que pode ser utilizada na sua empresa. Ele traz uma série de características interessantes, principalmente se você utiliza metodologias agile no setor de TI da sua empresa.

Isso porque ele possibilita a gestão de projetos por meio de SCRUM ou CMMI. Também permite a utilização de forma centralizada ou distribuída, sendo adequado tanto para equipes que compartilham o mesmo espaço físico quanto aquelas que trabalham à distância. Outra vantagem apresentada por esse sistema é o fato de não possuir limitações de crescimento e ter integração direta com o Microsoft Office.

MÓDULO V

4. GIT

O GIT é uma das ferramentas de controle de versão de software mais populares, principalmente em projetos open source. Isso se deve, principalmente, pela popularidade do GitHub, uma plataforma para hospedagem de códigos. Apesar de ela poder ser utilizada em outras ferramentas, o GIT é a que ganhou mais repercussão nesse aspecto.

As principais vantagens dessa ferramenta são o design interno e interface, a eficácia e o desempenho do software. Isso significa que ele é agradável de ser utilizado, consegue atingir todos os objetivos de um bom controle de software e é rápido. Porém, apresenta desvantagens relacionadas à simplicidade de utilização.

Apesar de ser a principal ferramenta de controle de versão de software disponível no mercado, possui controles um pouco mais complexos quanto comparado a outros softwares.

Isso significa que os colaboradores precisam entender uma série de conceitos mais profundos, relacionados ao controle de versão de software, para utilizar essa ferramenta corretamente. Apesar disso, é uma ferramenta de controle de versão distribuída, o que significa que é adequado para a utilização em grandes equipes, nas quais os desenvolvedores não estão localizados geograficamente no mesmo local.

MÓDULO V

5. Mercurial

O Mercurial é a ferramenta de controle de versão de software utilizada por grandes empresas como o Facebook e Google. Ela é bastante eficiente, o que significa que consegue desempenhar bem as funções básicas de um bom controle de software.

O Mercurial é uma ferramenta bastante rápida na execução dos comandos e ainda funciona muito bem para equipes grandes, nas quais os desenvolvedores não estão todos trabalhando no mesmo local.

Isso porque ela é uma ferramenta de controle de versão distribuída.

Essa ferramenta não apresenta muitas desvantagens apontadas por críticos. Ela é um pouco mais complexa de ser utilizada em comparação com a Subversion, por exemplo.

Porém, ainda é de aprendizagem fácil e rápida pelas equipes de desenvolvedores e possui medidas de segurança para impedir erros.

Agora você sabe quais são as principais características de 5 das melhores ferramentas de controle de versão de software. A partir disso, é importante que você entenda também quais são as principais necessidades e demandas da sua empresa. Ao cruzar esses dois dados, é possível identificar qual a melhor ferramenta de acordo com a realidade do seu time de TI.

MÓDULO V

Versionamento de código: o que é, como funciona e como fazer?

O versionamento de código se trata do **gerenciamento de mudanças em códigos e scripts**. Durante o desenvolvimento de uma aplicação tecnológica, correções e novas implementações são criadas diariamente. Por isso, é necessário trabalhar com o auxílio de ferramentas que ajudem no registro dessas modificações, indicando quem fez a mudança, quando e o que foi alterado.

Como funciona?

O versionamento de código é realizado durante a construção do mesmo. Para isso, é preciso acessar o terminal do ambiente de desenvolvimento, em que serão informados os comandos necessários, e ter uma ferramenta de versionamento instalada. A ferramenta mais popular que temos no mundo do desenvolvimento é o Git.

MÓDULO V

Essa ferramenta se comunicará com um repositório remoto de códigos, no caso do **Git** poderá ser o **GitHub**. Esse repositório remoto se trata de uma plataforma, que além de armazenar o código também oferece informações sobre as mudanças realizadas, quem as fez e quando.

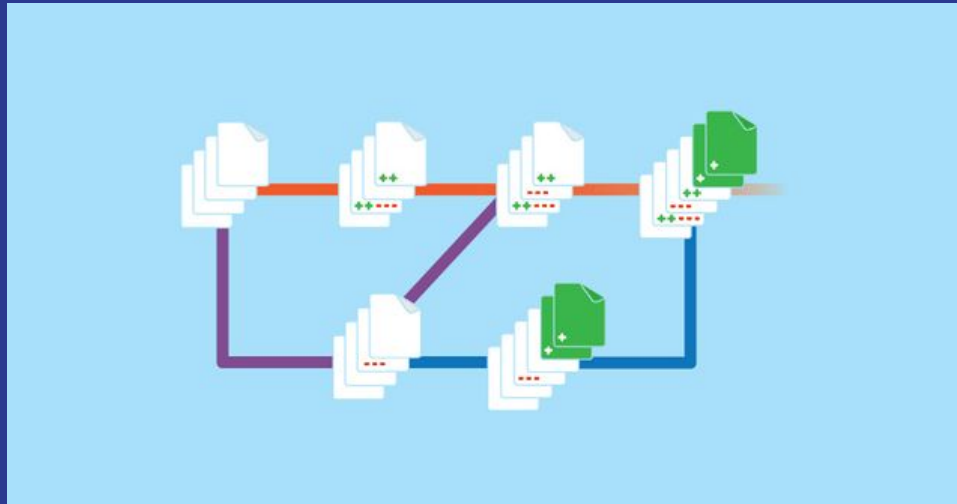
Em geral, as ferramentas de versionamento possuem uma **branch** (*ramo*) principal, que pode se chamar “**master**” ou “**main**”. Essa **branch** nada mais é que uma pasta, que além de guardar os arquivos de código, também possui informações sobre as mudanças feitas.

O código principal da aplicação é armazenado nela.

A partir do **ramo** principal podem ser criados outros. Esses **ramos** secundários vão conter uma cópia do código principal, que poderá ser modificado sem que o original seja alterado. Após as mudanças necessárias serem feitas, é possível solicitar permissão para que o código do ramo secundário se junte ao primeiro e eles se tornem um só.

MÓDULO V

Além disso, durante o versionamento das mudanças, também podemos adicionar comentários sobre o que foi modificado. Esses comentários são chamados de **commits** e são fundamentais para compreender o que foi realizado.



MÓDULO V

Quais as vantagens de fazer?

1. Segurança

Graças ao versionamento, é possível **recuperar arquivos, trechos de código e até versões completas de um projeto**. Este ponto é extremamente útil, pois garante a segurança do código. Além disso, apenas pessoas autorizadas poderão acessar o local de armazenamento.

2. Controle das versões

Essas ferramentas criam um **histórico das alterações realizadas**. Desse modo, é possível restaurar o código para um determinado momento, anterior a uma mudança. Isto é extremamente útil em caso de erros críticos, que inviabilizam o uso do sistema, ou em mudanças drásticas de escopo.

MÓDULO V

3. Rastreio das mudanças

Com o versionamento, é possível **identificar rapidamente o que foi alterado, quando e por qual pessoa**. Permitindo não só o rastreio das modificações, mas também a possibilidade de tirar dúvidas diretamente com a pessoa responsável.

4. Maior organização

O versionamento de código permite que os arquivos sejam armazenados de forma organizada. Para cada nova correção ou funcionalidade desenvolvida, é possível **criar uma pasta específica para armazenar estes arquivos, sem que haja interferência no código principal**.

Após validações e testes, o código pode então ser incorporado ao originário.

Outro ponto positivo, é que o histórico de mudanças é montado em ordem cronológica, gerando uma linha do tempo com as alterações feitas. Isso facilita a busca e visualização das modificações.

MÓDULO V

5. Colaboração

As ferramentas de versionamento facilitam também a colaboração. A partir delas, **outras pessoas podem mexer no código simultaneamente**, trabalhando em ramificações que derivam do código original, mas não o alteram diretamente. Como dito anteriormente, somente após validações essas mudanças poderão se juntar aos arquivos principais.

MÓDULO V

5 dicas para fazer o Versionamento de código

1. Crie commits objetivos

Os commits descrevem o que foi realizado na alteração que você criou. Por isso, é fundamental **descrevê-los com informações relevantes**. Criar comentários como “correção de bug” e “nova funcionalidade”, sem maiores detalhes, não ajudará outras pessoas a compreenderem o que foi feito.

2. Crie branches com nomes claros

Garanta que a ramificação que você criou possui um nome significativo. Assim, em projetos muito grandes e com várias pessoas atuando, será mais fácil encontrar o que você modificou e rapidamente entender qual parte do código foi alterada. Para exemplificar, em um sistema que possui diversos formulários de cadastros, criar um ramo chamado “correcao-erro-cadastro” não ajuda a entender qual deles foi modificado.

MÓDULO V

3. Não acumule muitas alterações

O versionamento de código existe como uma **garantia de que em caso de perdas ou erros críticos o código poderá ser recuperado**. Dessa forma, imagine que você tenha escrito mais de 200 linhas de código e seu computador pifa. Como as mudanças não haviam sido salvas no versionamento, todo o esforço foi pelo ralo. Por isso, lembre-se de gerar versões de tempos em tempos para que isso não aconteça.

4. Mantenha a branch principal atualizada

É a partir da ramificação principal que vamos criar outras aos quais iremos realizar nossas alterações. Por isso, é necessário mantê-la sempre atualizada. Assim, teremos a versão mais recente do código e evitaremos conflitos com alterações existentes, mas às quais não estavam presentes no código de trabalho.

MÓDULO V

5. Faça revisão de código

Um potente recurso disponível em diversas ferramentas de versionamento de código é o **Code Review** (Revisão de Código). Nesse caso, você poderá acionar uma outra pessoa autorizada a acessar o projeto para revisar as suas alterações e garantir que as mudanças poderão ser incorporadas ao código principal. Isso reduz o número de bugs e de código despadronizado.

MÓDULO V

Versionamento de software: o que é e como funciona?

O versionamento de software se trata da **criação de pacotes completos de código de uma aplicação**. Cada vez que uma alteração é feita e a aplicação inteira é compilada, gerando um pacote de códigos, uma nova versão do software surge.



MÓDULO V

Como funciona?

O versionamento de software é o passo posterior ao versionamento de código. Depois que os diversos trechos de código de uma aplicação são testados e aprovados, é possível gerar um pacote completo de arquivos, chamado de “build”. Esse pacote será disponibilizado em um servidor, para que outras pessoas externas ao projeto possam acessar o sistema criado.

A cada build, uma nova versão do software é gerada. Isso ocorre pela presença de novos trechos de código no pacote ou de arquivos que sofreram modificações. Geralmente, as mudanças na versão de um software são evidenciadas por números que acompanham o nome do sistema. Abordaremos este ponto com mais detalhes no tópico sobre versionamento semântico.

MÓDULO V

Quais as vantagens de fazer?

1. Correção de bugs

Erros, sejam eles críticos ou não, implicam em mudanças para que possam ser corrigidos. Desse modo, o versionamento de software é importante não só para documentar essas correções, mas também para **auxiliar na recuperação de uma versão em caso de um bug crítico**, em que o software se torne inutilizável.

2. Ajustes no layout

Seja por uma decisão da área de negócios ou por questões de usabilidade, **o layout de um software é algo que pode mudar com frequência**. Nesse sentido, é fundamental versionar o sistema para acompanhar essas alterações.

MÓDULO V

3. Mudanças na arquitetura

A área de tecnologia inova constantemente. Desse modo, para que os softwares não se tornem obsoletos é preciso acompanhar esta evolução. Por isso, o versionamento permite uma **evolução ordenada e organizada**, sem que o código que roda no momento seja impactado diretamente por uma nova mudança.

4. Facilita a colaboração

O versionamento de software, assim como o de código, permite que diversas pessoas **trabalhem em uma mesma base simultaneamente**, sem que afetem diretamente o trabalho umas das outras, o que acelera o processo de desenvolvimento.

MÓDULO V

5. Facilita a comunicação

Ao versionar um software, um documento pode ser criado para **atrelar a versão a um descritivo sobre o que foi modificado**. Isso traz mais transparência entre as pessoas desenvolvedoras e a área de negócios.

MÓDULO V

5 dicas para fazer o Versionamento de software

1. Documente as versões

Apenas mudar a versão não é esclarecedor o suficiente para saber o que mudou. Desse modo, é importante também **criar uma documentação que descreva o que foi alterado** em cada uma das versões.

2. Use padrões de desenvolvimento

Padrões de desenvolvimento **facilitam o entendimento do sistema** pela equipe. Além disso, garanta que as versões terão o mesmo modo de desenvolvimento.

MÓDULO V

3. Não acumele muitas mudanças

Lançar versões que contenham muitas mudanças podem assustar as pessoas usuárias. Isso se deve ao fato de que uma versão nova pode ter o layout ou o modo de executar uma funcionalidade modificados, impactando no conhecimento que a pessoa tinha do sistema e sendo necessário reaprender a utilizá-lo.

4. Faça testes

Mesmo que a base de código já tenha sido testada, alguns erros e inconsistências só são identificados quando o sistema é disponibilizado ao público. Sendo assim, **testar novamente o funcionamento é essencial para gerar builds melhoradas.**

MÓDULO V

5. Acompanhe o mercado

Às vezes, podemos nos deparar com sistemas que parecem ter parado no tempo, seja em relação ao layout, compatibilidade com hardwares e ferramentas mais atuais ou em questão de funcionalidade.

Por isso, tenha em mente que o fato de **um sistema ainda funcionar não significa que não deve ganhar novas versões**. Não atualizar um software pode torná-lo obsoleto e dispersar as pessoas usuárias.

MÓDULO V

Versionamento semântico: o que é e como funciona?

O versionamento semântico representa um **conjunto de regras utilizadas para gerenciar a nomeação de versões de software**. Tratam-se de números inteiros acrescidos ao nome do sistema, cada número informa o nível de modificação realizada na versão.

Como funciona?

Imagine que temos o “Software.2.5.33”. **O primeiro número da sequência (chamado de major) representa grandes alterações realizadas**. Isso significa que a nova versão se tornou incompatível com as anteriores.

O segundo número se refere a alterações médias (minor). Nesse caso, trata-se da incorporação de novas funcionalidades ao software. Por fim, **o último número representa as correções** realizadas (path).

MÓDULO V

O valor de cada item representa o número de vezes em que essas alterações aconteceram. Em nosso exemplo, podemos notar que o software sofreu duas grandes alterações e sua versão atual já não é mais compatível com a primeira.

Além disso, foram adicionadas cinco novas funcionalidades e identificados trinta e três pontos de correção. Além dessas questões, há ainda algumas premissas que devem ser seguidas para garantir maior controle da semântica entre os projetos, veja a seguir as principais:

- **Cada versão é única:** havendo modificações, uma nova versão deverá ser criada e os números atualizados de modo apropriado;
- **Seguir o padrão de três números inteiros:** a numeração de uma versão oficial não pode conter números negativos, letras ou zeros a esquerda;
- **Resetar path e minor quando houver mudança na major:** quando uma versão nova for lançada, tornando-se incompatível com a anterior, os números que se referem as funcionalidades e correções devem ser zerados.

MÓDULO V

Quais as vantagens de fazer?

O versionamento semântico garante organização e ajuda a comunicar atualizações às pessoas usuárias. Imagine um sistema que cresce rapidamente, conforme novas implementações e correções são acrescentadas, pode se tornar difícil encontrar rapidamente em qual versão da aplicação uma determinada mudança foi feita.

Por isso, usar o versionamento semântico pode auxiliar a criar documentações que relacionem os números da versão às alterações realizadas. Outro ponto positivo é a facilidade em comunicar que um sistema ou aplicativo está desatualizado. Ao apresentar o “**Software.1.2.15**” e o “**Software.1.3.18**”, a pessoa usuária compreenderá que houve alguma mudança, por conta da diferença nos números.

MÓDULO V

3. Use apenas números inteiros

O versionamento semântico funciona utilizando números inteiros. Por isso, uma versão oficial **não poderá conter números negativos, letras, caracteres especiais ou zeros ao lado esquerdo.**

4. Inicie a contagem a partir do número 1

Uma versão estável deve iniciar sua contagem como 1.0.0. Do mesmo modo, quando uma funcionalidade nova for acrescentada ou uma correção for realizada, elas também deverão iniciar a contagem em 1. Quando o número major for atualizado, os demais deverão ser zerados, pois trata-se de uma versão que já não é mais compatível com a anterior.

MÓDULO V

5. Documente o que foi alterado

Trocar os números das versões indicam que alterações foram feitas, mas os números, por si só, não dizem nada sobre a alteração. Por isso, é também preciso **criar uma documentação descrevendo os pontos que foram modificados** em cada versão nova.

MÓDULO V

Versionamento de documentos: o que é e como funciona?

Trata-se do **registro das sucessivas iterações realizadas em um arquivo de texto**.

O controle de versão de documentos é fundamental entre pessoas e corporações que trabalham compartilhando arquivos de textos que são alterados simultaneamente e com frequência.

Como funciona?

O controle de versão de documentos **pode ser feito por uma ferramenta ou por um conjunto de regras pré-definidas pelas pessoas envolvidas**, No caso da ferramenta, ela irá registrar uma nova alteração a cada caractere inserido, informando a data, hora e a pessoa que realizou a modificação.

Por outro lado, quando se trata de poucas pessoas envolvidas na modificação do arquivo, regras pontuais podem ser utilizadas. Nesse caso, pode-se definir uma estrutura de pastas, que irá guardar as diferentes versões do documento e uma numeração, indicando a versão.

MÓDULO V

Quais as vantagens de fazer?

1. Segurança

Quando você utiliza uma ferramenta de versionamento de documentos, além de garantir que o arquivo ficará salvo, também há a garantia de que **apenas as pessoas autorizadas poderão acessar e editar o documento.**

2. Criar versões secundárias

Assim como no versionamento de código, ao aplicar o versionamento de documentos é possível **criar rascunhos do documento principal.** Isso ajuda na validação de ideias e até de capítulos de livros.

MÓDULO V

3. Organização

Além de consultar quem fez mudanças no arquivo e quando, você poderá **recuperar versões anteriores**, saber como era o documento original e criar versões específicas para que apenas algumas pessoas ou setores acessem.

MÓDULO V

5 dicas para fazer o Versionamento de documentos

1. Crie critérios de aceitação

Para que o versionamento seja feito de forma eficiente, é preciso que haja um **padrão na criação das versões**. Por isso, estabeleça regras sobre como o documento deve ser criado e como ele deve ser versionado.

2. Documente as regras utilizadas

Quando falamos sobre os critérios de aceitação, não basta pensar que todas as pessoas envolvidas compreenderam tudo claramente ou que anotaram para revisar mais tarde. Por isso, crie um **arquivo contendo as regras que as versões do documento devem seguir e compartilhe** com as pessoas do time.

MÓDULO V

3. Faça um checklist

Quando finalizar a revisão ou escrita de um documento, tenha em mãos um checklist, contendo os critérios de aceitação citados anteriormente. Isso fará com que as versões sejam **analisadas com maior consistência e padronização**.

4. Salve na nuvem

Computadores podem apresentar problemas e caso você possua algum documento importante nele, pode ser que se torne irrecuperável. Por isso, na redação de um documento, tenha uma cópia também na nuvem, para garantir que o trabalho não será perdido.

MÓDULO V

4. Utilize editores online

Editores online, como o Google Docs, já possuem integração com um sistema de versionamento. Dessa forma, além de **compartilhar arquivos** com outras pessoas e poder **colaborar em outros textos**, o **histórico de mudanças** já criado a cada inserção e o documento fica salvo na nuvem.

Conclusão

O versionamento é uma forma organizada de administrar mudanças. Como vimos, as ferramentas de controle de versão podem ser aplicadas para gerenciar o ciclo de vida de documentos, softwares e códigos. Além disso, regras de controle de mudanças também podem ser aplicadas para gerenciar a numeração de uma versão.

MÓDULO V

Os conceitos do Git

São vários conceitos dentro do **Git** que nos permitem manipular o código-fonte dentro de suas inúmeras versões. Vamos listar os principais deles para que você possa se familiarizar.

Repositório

O primeiro é o repositório, local físico onde os arquivos e suas cópias ficarão armazenados. O repositório pode ser local ou remoto, podendo salvar não apenas arquivos de texto, mas também imagens, áudios e outros elementos relacionados ao projeto.

MÓDULO V

Branch

Branches são os ramos, cópias do código original que podem ser manipuladas de forma livre pela pessoa que trabalha em programação, sem afetar as funcionalidades em produção no código-fonte. Isso **permite que todas as alterações sejam realizadas de forma segura**, sem que erros ocorram na cópia principal do projeto.

Merge

Após a finalização de um trabalho em um Branch, é necessário realizar o Merge, fundir a cópia e seus arquivos modificados com o ramo principal do projeto. Isso acontece apenas no repositório local para que ajustes possam ser feitos.

MÓDULO V

Push Request

O Push Request é o envio das modificações após o Merge para o repositório central, para que todas as outras pessoas que atuam no desenvolvimento possam atualizar suas cópias e revisar o código criado, verificando conflitos com seus próprios trabalhos.

Pull Request

O Pull Request é utilizado quando outra pessoa que atua no desenvolvimento muda o ramo principal no repositório central, puxando as modificações realizadas para a sua máquina, fundindo a nova versão com o seu código local.

MÓDULO V

Fork

Trata-se do comando de cópia de um repositório remoto para a máquina local, realizado sempre que vamos começar a trabalhar em um projeto que já existe. Também é usado para pegar um código público para posterior modificação e utilização em um programa interno.

MÓDULO V

Como instalar e utilizar o Git

Ok, você já entendeu o conceito de **Git** e **GitHub**. Então, vamos por a mão na massa e mostrar na prática como criar um repositório **Git** e fazer suas primeiras alterações.

O primeiro passo é fazer o download do **Git** em sua máquina e realizar a instalação da ferramenta. Ele tem uma interface gráfica, além da integração com as principais IDEs do mercado. Porém, vamos fazer tudo em interface de comando.

Após instalar o **Git**, abra o prompt e utilize o seguinte comando:

```
git --version
```

MÓDULO V

Caso tudo tenha ocorrido conforme o esperado, o Git responderá mostrando a versão instalada em sua máquina.

Agora vamos criar a sua assinatura Git:

```
git config --global user.name "Seu usuario"  
git config --global user.email "email@example.com"
```

Pronto! Sempre que você salvar alguma alteração sua assinatura será incluída.

MÓDULO V

Para utilizar o **Git**, precisamos de uma pasta de projeto. Então, você pode entrar em seu diretório e criar uma pasta com o nome “*ProjetoTeste*” e, dentro dela, um arquivo txt com o nome “*teste.txt*”.

Escreva qualquer coisa nele.

No prompt de comando, navegue até dentro da pasta “*ProjetoTeste*” e dê o seguinte comando para a criação do repositório:

```
git init
```

MÓDULO V

A saída esperada é:

```
Initialized empty Git repository in C:/Users/SEU_USUARIO/ProjetoTeste/.git/
```

Já temos o nosso repositório Git, mas ele ainda está vazio, pois não salvamos nenhuma cópia.
Você pode verificar isso realizando o seguinte comando:

```
git status
```

MÓDULO V

A resposta em um repositório vazio como esse será:

```
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)

teste.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Ou seja, estamos no **branch master**, (ramo principal).

Não há alterações salvas, existe um arquivo que ainda não tem cópia e não há nada pronto para ser arquivado.

MÓDULO V

Para realizar a cópia desse novo arquivo, vamos utilizar dois comandos. O primeiro é:

```
git add teste.txt
```

Sendo que a saída esperada é:

```
Changes to be committed:  
  (use "git rm --cached <file>..." to unstage)  
  
new file: teste.txt
```

MÓDULO V

Significa que nosso arquivo já está na staged area, pronto para a criação de uma nova versão de código. O segundo comando, para confirmar e salvar essa alteração é:

```
git commit teste.txt
```

```
[master (root-commit) 690845c] Qualquer coisa escrita no arquivo  
1 file changed, 1 insertion(+)  
create mode 100788 teste.txt
```

Assim, caso utilizemos o comando de Git status novamente para verificar nossos arquivos, nossa resposta será um pouco diferente, já que o repositório não é mais vazio.

MÓDULO V

Como criar uma conta no Github?

1 Abra a página <https://github.com/join/plan?plan=free> em um navegador. Dá para usar qualquer navegador do seu computador, celular ou tablet.

- Algumas extensões que bloqueiam anúncios, como o uBlock Origin, impedem a verificação CAPTCHA. Caso passe pelo problema, desabilite a extensão para fazer o registro.

MÓDULO V

2 **Coloque seus dados pessoais.** Além de criar um nome de usuário (Username) e informar um endereço de e-mail válido (Email address), você tem que inserir uma senha (Password). A senha deve ter pelo menos 15 caracteres *ou*, se tiver no mínimo um número e uma letra minúscula, oito. ^[1]

- Leia com atenção os **Termos de Serviço** e a **Política de Privacidade** antes de prosseguir (ambos estão em inglês). Ao dar continuidade ao processo de cadastro, você diz que concorda com os dois documentos.

3 **Clique no botão azul** . Fica na parte de baixo do formulário.

MÓDULO V

- 4** **Faça a verificação CAPTCHA.** As instruções variam de acordo com o que é solicitado. Siga as orientações para confirmar que é uma pessoa e não um bot.
- Se você receber uma mensagem que diga "Não foi possível verificar a sua resposta captcha" é porque a extensão que bloqueia anúncios impediu a verificação CAPTCHA de aparecer. Desative todas as extensões desse tipo, atualize a página e clique em **Verifique** para começar.

MÓDULO V

5 Na tela de escolha de plano, selecione aquele que satisfizer melhor as suas necessidades. Depois de escolhê-lo, o GitHub envia uma mensagem de confirmação por e-mail. As opções de planos são:^[2]

- **Free:** repositórios públicos e privados ilimitados, até três colaboradores, fórum de problemas (issues), monitoramento de bugs e ferramentas de gerenciamento de projetos.
- **Pro:** acesso ilimitado a todos os repositórios, número ilimitado de colaboradores, fórum de problemas (issues), monitoramento de bugs e ferramentas avançadas.
- **Team:** todas as funcionalidades mencionadas anteriormente, controle de acesso de equipe e gerenciamento de usuários.
- **Enterprise:** todas as funcionalidades do plano Team, hospedagem própria ou em nuvem, prioridade na prestação de suporte técnico, login único (single sign-on) e mais.

MÓDULO V

6 Clique no botão  na mensagem de e-mail enviada pelo **GitHub**. Assim você confirma o endereço de e-mail e termina o processo de cadastro.

7 Revise a sua escolha de plano e clique em . Você pode indicar que quer ou não quer receber atualizações do site por e-mail marcando ou desmarcando a caixa de seleção "Send me updates".

- Caso tenha selecionado um plano pago, insira as informações de pagamento quando for solicitado.

MÓDULO V

8

Marque as suas preferências e clique em . O GitHub mostra um pequeno questionário para que o site se adapte às suas necessidades. Depois de fazer as escolhas, você é redirecionado à página do seu primeiro repositório.

- Se você quiser contratar um plano pago no futuro, basta clicar no último botão do canto superior e direito e acessar **Settings > Billing**.^[3]

Dicas

- Para entrar no seu painel de usuário, clique no ícone de um gato no canto superior e esquerdo da página.
- Para personalizar o perfil, clique no último botão do canto superior e direito da página e depois em **Your profile**.

MÓDULO V

EXERCÍCIOS...

MÓDULO V

FIM...